

Codalyra-AI

Backend platform for creating, running, validating, and evaluating AI coding-agent tasks in isolated, reproducible environments.

8	16	21	6
MODELS	ENDPOINTS	TESTS	PHASES

v0.1.0 Python 3.12 FastAPI + SQLAlchemy + PostgreSQL August 2026

CONTENTS

- | | |
|------------------------|-------------------------|
| 1. Project Overview | 7. Authentication |
| 2. System Architecture | 8. Technology Stack |
| 3. Project Structure | 9. Configuration |
| 4. Folder Workflows | 10. Development Roadmap |
| 5. Data Model | 11. Future Vision |
| 6. API Reference | 12. Getting Started |

1. Project Overview

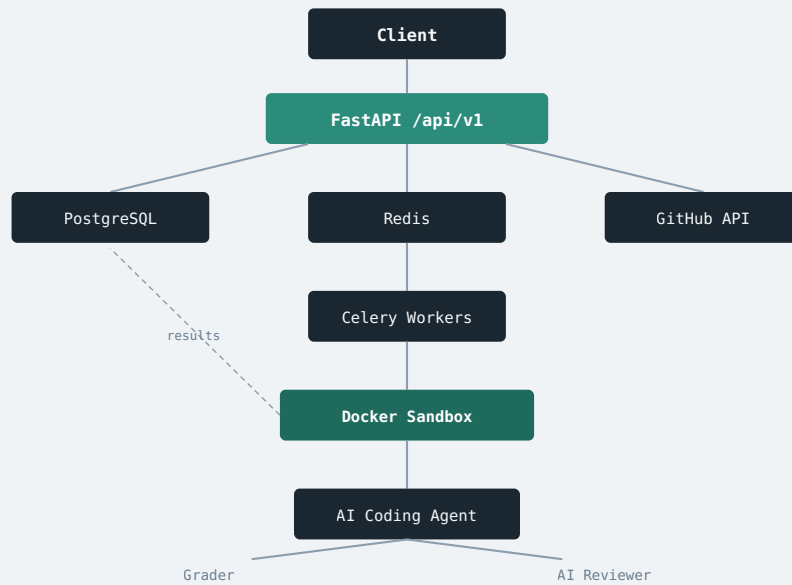
Real software-engineering tasks are rarely single-step problems. An AI coding agent may need to read an issue, understand the repository, find and modify code, run tests, debug failures, and submit a solution. Codalyra-AI provides the infrastructure to create these long-horizon tasks, execute them in isolated Docker environments, record the complete agent trajectory, and produce structured evaluations.

Core Capabilities

- **Task management** — Create coding tasks with descriptions, difficulty ratings, test commands, time limits, and evaluation criteria
- **Repository integration** — Connect tasks to Git repositories with exact commit-level reproducibility
- **Agent execution** — AI agents interact through tool calls: `read_file`, `write_file`, `run_command`, `run_tests`, `search_code`, `git_diff`
- **Trajectory recording** — Every agent action is captured as a step in the trajectory
- **Checkpoint & resume** — Long-running tasks can be checkpointed and resumed after failures
- **Automated validation** — Language-specific validators for Python and Go
- **Structured grading** — Multi-criteria evaluation with partial-credit scoring
- **Hidden tests** — Tests the agent never sees, preventing solution-fitting
- **Docker isolation** — Each run executes in its own container

2. System Architecture

The system follows a layered architecture with clear separation between API, business logic, data access, and infrastructure.



System architecture – request flow from client to evaluation

Dependency Chain

config ← database/session ← models ← repositories ← services ← routes ← main

3. Project Structure

`codalyra-ai/` pyproject.toml Project config & dependencies `docker-compose.yml`
 PostgreSQL + Redis `alembic.ini` Migration config `alembic/` Migration runner & versions
`app/` `main.py` FastAPI entry point `config.py` Settings (pydantic-settings)
`dependencies.py` Auth dependencies `api/v1/` Route handlers (6 files) `core/` Security, JWT, exceptions `database/models/` 8 SQLAlchemy models `database/` Base, session, engine
`schemas/` Pydantic schemas (7 files) `repositories/` Data access layer (4 repos)
`services/` Business logic (8 services) `workers/` Celery background tasks `ai/` LLM integration
`github/` GitHub API client `validators/` Code validators `utils/` Helpers, logger, constants
`scripts/` Admin & seed scripts `tests/` Unit & integration tests

4. Folder Workflows

Each folder has a specific role in the request lifecycle. This section explains what every folder does, what files it contains, and how data flows through it.

app/ ACTIVE

Application root. Contains the FastAPI entry point and top-level configuration. Every request starts here.

<code>main.py</code>	Creates the FastAPI app, registers exception handlers, includes all routers
<code>config.py</code>	Loads env vars via pydantic-settings into a global <code>Settings</code> object
<code>dependencies.py</code>	OAuth2 bearer scheme, <code>get_current_user</code> dependency for protected routes

Request → `main.py` → router dispatch → dependency injection (auth, db session) → route handler

app/api/v1/ ACTIVE

HTTP route handlers. Each file defines an `APIRouter` for a resource. Routes validate input via Pydantic schemas, inject dependencies (DB session, current user), delegate to services, and return responses. No business logic lives here.

<code>__init__.py</code>	Aggregates all routers into <code>api_router</code> with prefix <code>/api/v1</code>
<code>health.py</code>	<code>GET /health</code> , <code>GET /health/db</code> — liveness and database connectivity checks
<code>auth.py</code>	<code>POST /auth/register</code> , <code>POST /auth/login</code> , <code>GET /auth/me</code> — public auth endpoints
<code>users.py</code>	<code>GET /users/{id}</code> , <code>PUT /users/{id}</code> — user profile management
<code>projects.py</code>	Full CRUD for projects — create, list (by owner), get, update, soft-delete
<code>repositories.py</code>	Full CRUD for repositories — scoped to projects via <code>?project_id=</code>
<code>tasks.py</code>	Full CRUD for tasks — scoped to projects via <code>?project_id=</code>
<code>github.py</code>	Phase 2 — GitHub webhook receiver and repo sync endpoints

Client `POST /api/v1/tasks/` → `tasks.py` validates `TaskCreate` schema → injects `db` + `current_user` → calls `task_service.create()` → returns `TaskResponse`

app/core/ ACTIVE

Cross-cutting concerns: authentication primitives, authorization, and error handling. Used by every layer but depends on nothing except config.

security.py	hash_password() and verify_password() using bcrypt directly
jwt.py	create_access_token() and decode_access_token() using python-jose HS256
exceptions.py	Exception hierarchy: CodaLyraException base, then NotFoundException (404), UnauthorizedException (401), ForbiddenException (403), ConflictException (409), BadRequestException (400)
permissions.py	Stub — role-based access control for future admin/team features

Register: password → security.hash_password() → stored in DB
Login: password → security.verify_password() → jwt.create_access_token() → returned to client
Protected route: token → jwt.decode_access_token() → user lookup → request proceeds or raises UnauthorizedException

app/database/ ACTIVE

Database engine, session factory, and ORM base class. Provides the get_db() dependency that injects a session into every request.

base.py	Defines Base(DeclarativeBase) — all models inherit from this
session.py	Creates engine from DATABASE_URL, SessionLocal factory, get_db() generator that yields a session and auto-closes

Request arrives → FastAPI calls get_db() → SessionLocal() creates session → session injected into route → route uses session for queries → session closed after response

app/database/models/ ACTIVE

SQLAlchemy ORM models that define database tables. Each model uses `mapped_column`, UUID primary keys, and timezone-aware timestamps. The `__init__.py` imports all models so Alembic can auto-detect them for migrations.

<code>user.py</code>	Users table — email, username, hashed_password, is_active, is_superuser
<code>project.py</code>	Projects table — name, description, owner_id FK, soft-delete via is_active
<code>repository.py</code>	Repositories table — name, url, language, default_branch, project_id FK
<code>task.py</code>	Tasks table — title, difficulty, status, time_limit, test_command, metadata (JSON)
<code>run.py</code>	Runs table — agent_name, status, timing, exit_code, error_message
<code>checkpoint.py</code>	Checkpoints table — run snapshots with sequence_number, file_path, diff_content
<code>trajectory.py</code>	Trajectories table — agent action log: action_type, input, output, duration
<code>evaluation.py</code>	Evaluations table — one per run: score, tests_passed/total, is_correct, feedback

Model change → alembic revision --autogenerate → migration script created → alembic upgrade head → table updated in PostgreSQL

app/schemas/ ACTIVE

Pydantic v2 models for request validation and response serialization. Each entity has Base (shared fields), Create (input), Update (partial), and Response (output with `from_attributes=True`) variants.

<code>auth.py</code>	LoginRequest, Token, TokenPayload
<code>user.py</code>	UserCreate (password min 8), UserUpdate, UserResponse
<code>project.py</code>	ProjectCreate, ProjectUpdate, ProjectResponse
<code>repository.py</code>	RepositoryCreate, RepositoryUpdate, RepositoryResponse
<code>task.py</code>	TaskCreate, TaskUpdate (with Literal types for difficulty/status), TaskResponse
<code>run.py</code>	RunCreate, RunResponse
<code>evaluation.py</code>	EvaluationCreate, EvaluationResponse

POST body → Pydantic validates against TaskCreate → rejects invalid (422) or passes clean data to service → service returns ORM object → Pydantic serializes via TaskResponse

app/repositories/ ACTIVE

Data access layer. Each repository encapsulates all SQL queries for one model. Receives a SQLAlchemy `Session`, returns ORM objects. No business logic — just queries.

user_repository.py	get_by_id, get_by_email, get_by_username, create, update, list_all
project_repository.py	get_by_id, get_by_owner, create, update, delete (sets is_active=False)
task_repository.py	get_by_id, get_by_project, create, update, delete, count_by_project
run_repository.py	get_by_id, get_by_task, create, update

Service calls `repo.get_by_id(db, id) → repo` executes `db.query(Model).filter(...).first()` → returns ORM object or None → service decides what to do (raise 404, etc.)

app/services/ ACTIVE

Business logic layer. Services coordinate between repositories, enforce ownership rules, handle password hashing, JWT creation, and cross-entity validation. This is where "who can do what" is decided.

auth_service.py	<code>register()</code> : checks email/username uniqueness, hashes password, creates user. <code>login()</code> : verifies credentials, returns JWT
user_service.py	<code>get_user()</code> , <code>update_user()</code> , <code>list_users()</code>
project_service.py	CRUD with ownership check — only the project owner can update/delete
task_service.py	CRUD with ownership check — verifies user owns the parent project before allowing changes
repository_service.py	CRUD with project ownership verification
review_service.py	Stub — will orchestrate AI code review workflow
github_service.py	Stub — will handle GitHub repo sync and webhook processing
report_service.py	Stub — will generate evaluation reports and analytics
notification_service.py	Stub — will send Slack/email notifications on run completion

Route calls `project_service.update(db, project_id, data, current_user) → service` loads project via repo → checks `current_user.id == project.owner_id` → if no: raises `ForbiddenException` → if yes: `repo.update()` → returns updated project

app/workers/ PHASE 2

Celery background task workers. These run outside the request/response cycle for long-running operations like agent execution, report generation, and GitHub sync. Redis serves as the message broker.

<code>celery_app.py</code>	Celery application factory — configures broker (Redis), result backend, task serialization
<code>review_tasks.py</code>	Task: spin up Docker sandbox, execute agent run, record trajectory, produce evaluation
<code>github_tasks.py</code>	Task: clone repos, sync branches, process webhook events asynchronously
<code>report_tasks.py</code>	Task: aggregate evaluation results, generate comparison reports, compute rankings

API receives POST /runs → creates Run record (status=pending) → dispatches Celery task → worker picks up task → spins up Docker container → runs agent → records trajectory → evaluates → updates Run (status=completed)

app/ai/ PHASE 3

LLM integration layer. Manages API clients for AI models, prompt templates, code analysis, and AI-powered code review. Abstracts the LLM provider so the rest of the app doesn't care if it's OpenAI, Anthropic, or a local model.

<code>client.py</code>	Unified LLM client — wraps OpenAI SDK with retry logic, token counting, cost tracking
<code>prompts.py</code>	Prompt templates for code review, bug analysis, test generation, and architecture review
<code>analyzer.py</code>	Static code analysis: complexity scoring, dependency mapping, change impact assessment
<code>reviewer.py</code>	AI code reviewer: takes a diff, runs it through prompts, returns structured review with severity ratings
<code>embeddings.py</code>	Code embeddings for semantic search: find similar code, detect copy-paste, cluster related files

Evaluation phase → reviewer.py receives agent's final diff → builds prompt from prompts.py → calls client.py → LLM returns structured JSON review → stored as part of Evaluation record

app/github/ PHASE 2

GitHub API integration. Handles repository cloning, branch management, diff parsing, and webhook processing. Uses httpx for async HTTP calls to the GitHub REST API.

<code>client.py</code>	GitHub API client — authenticated requests, rate limiting, pagination handling
<code>parser.py</code>	Parses GitHub webhook payloads (push, PR, issue events) into internal event objects
<code>diff.py</code>	Git diff parsing and formatting — unified diff to structured file-level changes
<code>webhook.py</code>	Webhook signature verification, event routing, idempotency checks

GitHub sends webhook → `webhook.py` verifies HMAC signature → `parser.py` extracts event data → dispatches to `github_tasks.py` worker → worker clones/syncs repo → updates Repository record

app/validators/ PHASE 4

Language-specific code validators that run inside Docker sandboxes. Each validator knows how to check syntax, run tests, lint, and type-check for its language. Returns a structured report with pass/fail and detailed findings.

<code>python_validator.py</code>	Runs pytest (test results), ruff (linting), mypy (type checking) — parses output into scores
<code>golang_validator.py</code>	Runs go test, go vet, golangci-lint — handles Go module setup and build errors
<code>security_validator.py</code>	Scans for hardcoded secrets, SQL injection patterns, XSS vectors, unsafe file operations

Agent completes run → worker detects language from task config → selects `python_validator` → runs pytest inside sandbox → parses "5 passed, 2 failed" → runs ruff → parses warnings → `security_validator` checks diff → all results merged into Evaluation

app/utils/ ACTIVE

Shared utilities used across all layers. Small, stateless helper functions and application-wide constants.

<code>logger.py</code>	Configured Python logger with structured JSON output and log levels
<code>constants.py</code>	Application constants: status enums, difficulty levels, default values
<code>helpers.py</code>	Utility functions: UUID generation, timestamp formatting, string sanitization

app/scripts/ ACTIVE

One-off administration scripts for development and deployment setup.

<code>create_admin.py</code>	Creates a superuser account for initial system setup
<code>seed_database.py</code>	Populates the database with sample projects, tasks, and test data for development

app/tests/ ACTIVE

Test suite using pytest. Unit tests use SQLite in-memory with StaticPool for isolation. Integration tests (stubs) will hit real PostgreSQL. The `conftest.py` provides `db_session`, `client` (TestClient with dependency overrides), and `auth_headers` fixtures.

<code>conftest.py</code>	Test fixtures: in-memory SQLite engine, session factory, TestClient, authenticated headers
<code>unit/test_auth.py</code>	8 tests — register, login, duplicate email/username, wrong password, token validation
<code>unit/test_projects.py</code>	6 tests — create, list, get, update, delete, unauthorized access
<code>unit/test_tasks.py</code>	6 tests — create, list by project, get, update, delete, cross-project isolation
<code>unit/test_reviews.py</code>	1 test — health endpoint verification
<code>unit/test_ai.py</code>	Stub — will test AI reviewer with mocked LLM responses
<code>integration/</code>	Stub — <code>test_api.py</code> , <code>test_database.py</code> , <code>test_github.py</code> for full-stack integration tests

```
uv run pytest → conftest creates in-memory SQLite → creates all tables → test registers user, gets token → test calls API with TestClient → assertions → tables dropped after each test
```

alembic/ ACTIVE

Database migration framework. The `env.py` reads `DATABASE_URL` from app config and imports `Base.metadata` so Alembic can auto-detect model changes and generate migration scripts.

<code>env.py</code>	Migration runner — connects to DB, runs migrations in transaction
<code>versions/</code>	Auto-generated migration scripts (one per schema change)

```
Add column to model → alembic revision --autogenerate -m "add column" → script generated in versions/ → alembic upgrade head → ALTER TABLE runs on PostgreSQL
```

Complete Request Lifecycle

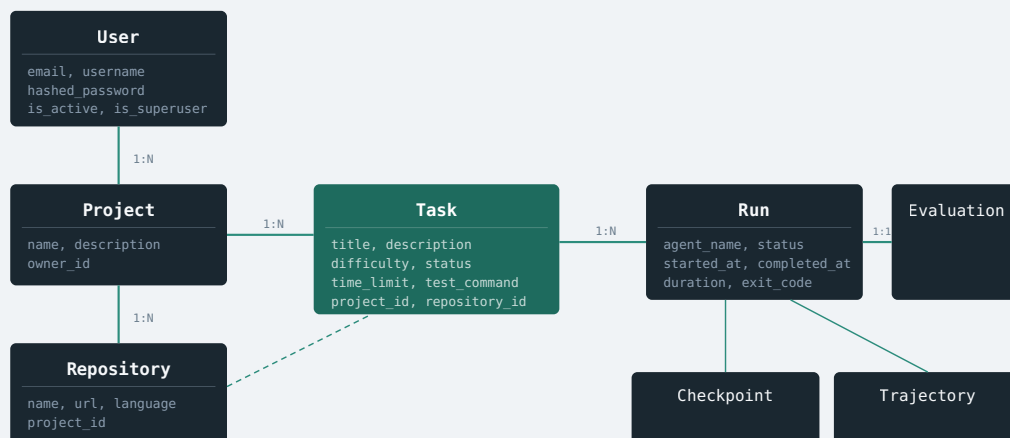
Here's how all folders work together for a typical authenticated API call:

1. Client sends `POST /api/v1/tasks/` with JWT and JSON body
2. `main.py` routes to `api/v1/tasks.py`

3. `dependencies.py` extracts JWT → `core/jwt.py` decodes it → `repositories/user_repository.py` loads user
4. `schemas/task.py` validates the request body against `TaskCreate`
5. `api/v1/tasks.py` calls `services/task_service.py.create()`
6. Service verifies user owns the project via `repositories/project_repository.py`
7. Service calls `repositories/task_repository.py.create()` with validated data
8. Repository creates a `database/models/task.py` ORM instance, commits to DB via `database/session.py`
9. `schemas/task.py` serializes the ORM object into `TaskResponse`
10. Response returned to client as JSON

5. Data Model

Eight models, UUID primary keys, timezone-aware timestamps, SQLAlchemy 2.0 mapped columns.



Entity-relationship diagram

Run / Checkpoint / Trajectory / Evaluation

MODEL	KEY FIELDS	RELATION
Run	task_id, agent_name, status, started_at, completed_at, duration, exit_code	Task 1:N
Checkpoint	run_id, sequence_number, file_path, content_hash, diff_content	Run 1:N
Trajectory	run_id, sequence_number, action_type, action_input, action_output, duration_ms	Run 1:N
Evaluation		Run 1:1

MODEL	KEY FIELDS	RELATION
	run_id (unique), tests_passed, tests_total, score, is_correct, feedback	

6. API Reference

All endpoints prefixed with `/api/v1`. Protected endpoints require

Authorization: Bearer <token>.

Health

GET	/health	Application health
GET	/health/db	Database connectivity

Authentication

POST	/auth/register	Create account
POST	/auth/login	Get JWT token
GET	/auth/me	Current user info

Users

GET	/users/{id}	Get user by ID
PUT	/users/{id}	Update user

Projects

POST	/projects/	Create project
GET	/projects/	List user's projects
GET	/projects/{id}	Get project
PUT	/projects/{id}	Update project
DEL	/projects/{id}	Soft-delete

Repositories & Tasks

Both follow identical CRUD patterns: POST `/`, GET `/?project_id=`, GET `/ {id}`, PUT `/ {id}`, DELETE `/ {id}`

7. Authentication

JWT bearer tokens, bcrypt password hashing, HS256 signing, 30-minute TTL.

1. POST /auth/register → Account created
2. POST /auth/login → Returns { access_token, token_type }
3. Authorization: Bearer <token>
4. GET /auth/me → Returns current user

8. Technology Stack

CATEGORY	TECHNOLOGY	PURPOSE
Language	Python 3.12	Primary runtime
Framework	FastAPI 0.141	Async REST API
ORM	SQLAlchemy 2.0	Declarative models
Validation	Pydantic 2.13	Schemas
Migrations	Alembic 1.19	Schema versioning
Database	PostgreSQL 16	Primary store
Queue	Redis 7 + Celery 5.6	Background tasks
Auth	python-jose + bcrypt	JWT + hashing
Containers	Docker + Compose	Isolation
Testing	pytest	Test suite
Packages	uv	Dependency mgmt

9. Configuration

VARIABLE	DEFAULT	DESCRIPTION
DATABASE_URL	postgresql://...localhost:5433/ codalyra	PostgreSQL connection
REDIS_URL	redis://localhost:6379/0	Redis connection
SECRET_KEY	change-me-in-production	JWT signing key
ALGORITHM	HS256	JWT algorithm
ACCESS_TOKEN_EXPIRE_MINUTES	30	Token TTL

10. Development Roadmap

Phase 1 — Backend Foundation COMPLETE

- FastAPI with layered architecture, PostgreSQL + SQLAlchemy with 8 models
- JWT auth, CRUD APIs, 21 tests, Docker Compose dev setup

Phase 2 — Task Execution NEXT

- Celery + Redis task queue, Docker sandbox provisioning, run management API
- Git integration, command execution, status streaming via WebSocket

Phase 3 — Agent System PLANNED

- Agent tool interface, trajectory recording, token/cost tracking
- Timeout handling, checkpoint and resume for long runs

Phase 4 — Evaluation PLANNED

- Python/Go/security validators, hidden tests, multi-criteria rubrics
- Partial-credit scoring, AI code reviewer

Phase 5 — Reliability PLANNED

- Checkpoint persistence, retry with backoff, worker locking, dead-letter queues
- Structured logging with correlation IDs

Phase 6 — Production PLANNED

- Multi-stage Docker build, CI/CD (GitHub Actions), Kubernetes manifests
- Prometheus + Grafana, health probes, Terraform IaC

11. Future Vision

High-value extensions beyond the core six phases.

ANALYTICS

Agent Leaderboard & Benchmarking

Elo-style ratings, pass@k metrics, cost-per-solve tracking, performance heatmaps by language/difficulty. Public and private leaderboards.

INTELLIGENCE

Trajectory Analytics & Failure Patterns

ML-powered analysis of trajectories to identify failure modes: infinite loops, hallucinated APIs, wrong-file edits. Cluster similar failures across runs.

AUTHORING

Task Generator from Real Repos

Auto-generate tasks from GitHub issues, PRs, and bugs. Use git history for before/after snapshots with ground-truth solutions.

ECOSYSTEM

Multi-Agent Collaboration Tasks

Tasks requiring multiple agents: one writes code, another reviews, a third writes tests. Evaluate inter-agent communication.

LANGUAGES

Polyglot Task Support

JS/TS, Rust, Java, C++ validators. Cross-language tasks (fix API + client together).

SECURITY

Red Team & Adversarial Tasks

Test agent safety with adversarial prompts. OWASP-aligned rubrics. Detect if agents propagate injected bugs.

PLATFORM

Plugin SDK for Custom Agents

Standardized SDK with OpenAI/Anthropic-compatible adapters. Define capabilities, permissions, resource limits.

INTEGRATION

CI/CD Pipeline Integration

GitHub Action / GitLab CI components. Gate merges on evaluation scores. Slack/Discord notifications.

REPLAY

Interactive Trajectory Replay

Web-based step-through debugger. Fork from any checkpoint. Side-by-side agent comparison on same task.

SCALE

Distributed Evaluation at Scale

Kubernetes auto-scaling, spot instances, batch evaluation API for running benchmark suites overnight.

12. Getting Started

Setup

```
cd codalyra-ai
cp .env.example .env
docker compose up -d
uv sync
uv run alembic upgrade head
uv run fastapi dev app/main.py
```

Verify

```
curl http://localhost:8000/api/v1/health
open http://localhost:8000/docs
uv run pytest
```

Commands

COMMAND	PURPOSE
uv run fastapi dev app/main.py	Dev server with hot reload
uv run pytest	Run test suite
uv run alembic upgrade head	Apply migrations
uv run alembic revision --autogenerate -m "desc"	Generate migration
docker compose up -d	Start infrastructure
docker compose down -v	Stop and remove data